Scalability & Future Plan

Date	July 9, 2026
Team ID	SWTID-2026-1262
Project Name	OptiCrop: Smart Agricultural Production Optimization Engine
Maximum Marks	1 Mark

Scalability & Future Plan:

Current System Limitations:

S.No	Limitation	Impact	Priority to Address (High / Medium / Low)
1	The Random Forest tree vectors are loaded entirely into RAM at server boot time.	Limits the model size and causes scaling overhead if the training dimensions increase significantly.	Medium
2	The app runs on Flask's built-in development network loop loopback gateway.	Unsuitable for heavy concurrent multi-user production traffic in real field conditions.	High
3	Crop sustainability guidelines and recommendations rely on static internal Python dictionaries.	Requires manual code redeployments to update regional ecological baselines or change threshold metrics.	Medium

Scalability Plan:

S.No	Scalability Aspect	Current State	Proposed Upgrade / Solution
1	User Load	Handles up to 50 concurrent simulation requests safely within local server bounds.	Wrap the Flask server configuration inside a robust production container layer (Docker + Gunicorn) managed by an Nginx reverse proxy.
2	Data Storage	Synthetic dataset matrices are stored in local .csv files; models are persisted in binary blobs.	Migrate regional soil telemetry and user lookup profiles into an external PostgreSQL relational database schema.
3	Performance	Delivers sub-400ms inference times on entry-level hardware specs for standalone arrays.	Implement Redis caching layers to store frequent regional soil profile queries and bypass the ML model loop for identical vectors.
4	Security	Basic float range constraints running on backend routing rules catch out-of-bounds metrics.	Integrate strict client-side encryption, API keys, and HTTPS transport protocols to protect proprietary farm telemetry logs.

Future Roadmap:

Phase	Planned Feature / Enhancement	Target Timeline	Expected Impact
Phase 2	IoT Edge Device Sensor Integration	3 Months	Enables automated, live field data transmission (N, P, K sensors) directly to the Flask app via REST API webhooks, eliminating manual data entry.
Phase 3	Dynamic External Weather API Integration	6 Months	Replaces manual temperature, humidity, and rainfall inputs with automated real-time local forecasts pulled directly from OpenWeatherMap APIs.
Phase 4	Geographic Information System (GIS) Mapping	12 Months	Renders Scenario 3 (Policy & System Analysis) recommendations onto interactive satellite layout maps to help planners visualize regional land health.